

WEEK 4

Module 2

Responsive Design

Media queries for phones, tablets and larger screens, and the practices that keep a responsive stylesheet maintainable.

2 topics · [CSS Study Notes](#)

Contents

- 01 Media Queries
- 02 Best Practices for Media Queries

01 Media Queries

Syntax · Breakpoints · Targeting phones, tablets and larger screens

The syntax

```
@media (min-width: 768px) {  
  .container { display: flex; }  
}
```

The rules inside the block apply only when the condition is true. A media query does not create new styles by itself; it switches a set of rules on at a particular size.

min-width against max-width

Query	Applies when	Approach
<code>(min-width: 768px)</code>	The viewport is 768px or wider	Mobile first: the base styles serve small screens and each query adds to them going upwards.
<code>(max-width: 767px)</code>	The viewport is 767px or narrower	Desktop first: the base styles serve large screens and queries subtract going downwards.

The two can be combined for a band that applies only between two sizes: `@media (min-width: 768px) and (max-width: 1023px)`.

A common breakpoint set

```
/* base styles: mobile, no query required */  
  
@media (min-width: 480px) { /* large phones      */ }  
@media (min-width: 768px) { /* tablets          */ }  
@media (min-width: 1024px) { /* small laptops    */ }  
@media (min-width: 1280px) { /* desktops        */ }
```

These figures are conventions rather than rules. The right breakpoint is the width at which your own layout begins to look wrong, which is covered in the next topic.

Beyond width

Query	Meaning
<code>(orientation: landscape)</code>	The viewport is wider than it is tall.
<code>(min-height: 600px)</code>	Useful for full-screen sections on short screens.
<code>(hover: hover)</code>	The device has a true hover capability, which excludes most touchscreens.
<code>(pointer: coarse)</code>	The primary input is imprecise, such as a finger, so tap targets should be larger.
<code>(prefers-color-scheme: dark)</code>	The user has chosen a dark interface.
<code>(prefers-reduced-motion: reduce)</code>	The user has asked for less animation.
<code>print</code>	A media type rather than a feature, used for print stylesheets.

Modern browsers also accept range syntax, which reads more naturally: `@media (width >= 768px)`.

The prerequisite in the HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

WITHOUT THIS TAG NOTHING WORKS

A mobile browser otherwise pretends to be about 980 pixels wide and shrinks the whole page to fit, so a query written for 768 pixels never matches and the site appears zoomed out. This single line in the head is what makes every media query behave as expected.

Combining conditions

```
@media (min-width: 768px) and (orientation: landscape) { }  
@media (max-width: 600px), print { }           /* comma means OR */  
@media not all and (min-width: 768px) { }       /* negation      */
```

QUICK RECAP

- min-width builds upwards from mobile; max-width narrows down from desktop.
- Queries can test orientation, hover, pointer type and user preferences, not just width.
- The viewport meta tag is required for any of it to take effect on a phone.

02 Best Practices for Media Queries

Mobile first · Content-driven breakpoints · Writing fewer queries

Start with mobile first

```
/* base: works on the smallest screen */
.grid { display: grid; grid-template-columns: 1fr; gap: 16px; }

@media (min-width: 768px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .grid { grid-template-columns: repeat(3, 1fr); }
}
```

Mobile-first is preferred for three reasons. A small screen forces the content hierarchy to be decided first. The base stylesheet stays simple, because it holds the simplest layout. And each query adds rules rather than undoing them, so the cascade works with you instead of against you.

Let the content choose the breakpoint

THE SINGLE MOST USEFUL HABIT

Do not begin with a list of device widths. Widen the browser slowly and watch for the point at which the layout starts to look wrong, such as a line of text becoming too long or cards becoming too narrow. Place the breakpoint there. Device-specific figures date quickly, whereas a breakpoint chosen from your own content stays correct.

Chasing individual devices leads to a stylesheet full of queries for a phone released three years ago, and a layout that still breaks on the next screen size nobody anticipated.

Write fewer queries by using modern CSS

```
/* a responsive grid with no media queries at all */
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 20px; }

/* fluid type with no breakpoints */
h1 { font-size: clamp(1.75rem, 4vw, 3rem); }

/* a container that caps itself */
.wrapper { width: min(1200px, 92%); margin-inline: auto; }
```

Each of these replaces several breakpoints, and each adapts continuously rather than only at fixed thresholds. Reach for a media query when the layout must genuinely **change shape**, such as a sidebar moving below the content, and let fluid values handle simple resizing.

Practical rules

- **Keep queries with their component.** Placing a component's breakpoints directly after its rules keeps related code together, which is easier to maintain than one block of queries at the end of the file.
- **Use `em` or `rem` in the query itself**, as in `@media (min-width: 48em)`. These respect the user's default font size, so the layout changes at the right moment for someone reading at a larger size.
- **Avoid overlapping ranges.** Mixing `min-width` and `max-width` queries for the same component creates conflicts that are hard to trace. Choose one direction and keep to it.
- **Order matters.** With a mobile-first approach the queries must run from smallest to largest, because a later rule of equal specificity wins.
- **Test by resizing, not only at the breakpoints.** Most layout faults appear in between, at the widths nobody thought to check.
- **Do not hide content to make it fit.** Using `display: none` on a mobile layout means those users lose information altogether; rearrange it instead.
- **Check tap targets.** Interactive elements should be at least around forty-four pixels in each direction, which `(pointer: coarse)` can help enforce.

Testing

Browser developer tools are sufficient for most work, but a real device behaves differently in two ways worth remembering: mobile toolbars change the usable height, which is why `dvh` is safer than `vh`, and touch has no hover state, so anything hidden behind a hover must have a tap alternative.

QUICK RECAP

- Build upwards from the smallest screen so that queries add rather than undo.
- Choose breakpoints from where your layout breaks, not from device names.
- Fluid tools such as clamp, min and auto-fit remove the need for many queries.